

점자 채팅 구현 프로젝트

2024171105 박예은
2022170401 박종현





🔍 목차

프로젝트 개요

코드 구조

출력 및 결과

결론

1. 프로젝트 개요 - 배경

점자는 시각장애인이 문자 정보를 인식하기 위한 가장 기본적인 수단

전자 기기에서 점자를 직접 입력, 실시간으로 변환해 출력하는 장치 보편화 X

하드웨어 레벨에서 점자 입력을 처리, 저장, 시각적으로 확인할 수 있게 설계

1. 프로젝트 개요

a 0097 	b 0098 	c 0099 	d 0100 	e 0101 	f 0102 	g 0103 
h 0104 	i 0105 	j 0106 	k 0107 	l 0108 	m 0109 	n 0110 
o 0111 	p 0112 	q 0113 	r 0114 	s 0115 	t 0116 	u 0117 
v 0118 	w 0119 	x 0120 	y 0121 	z 0122 		

1. 프로젝트 개요

I'm fine. Thank you. And you?

오전 10:47

Hi. How are you?

오전 10:47

Professor, I love you :)

1 오전 10:47

1. 프로젝트 개요

프로젝트 개요
: 3차 3차 3차

오전 10:47

오전 10:47

프로젝트 개요

1 오전 10:47

프로젝트 개요

2. 코드 구조 – TOP MODULE

입출력 모듈

버튼, 스위치를 이용한 입력 신호
LED, Text lcd, Tft lcd를 이용한 출력 신호

converter

각 버튼의 입력 신호 -> 6bit 점자 신호 -> 8bit 아스키 코드

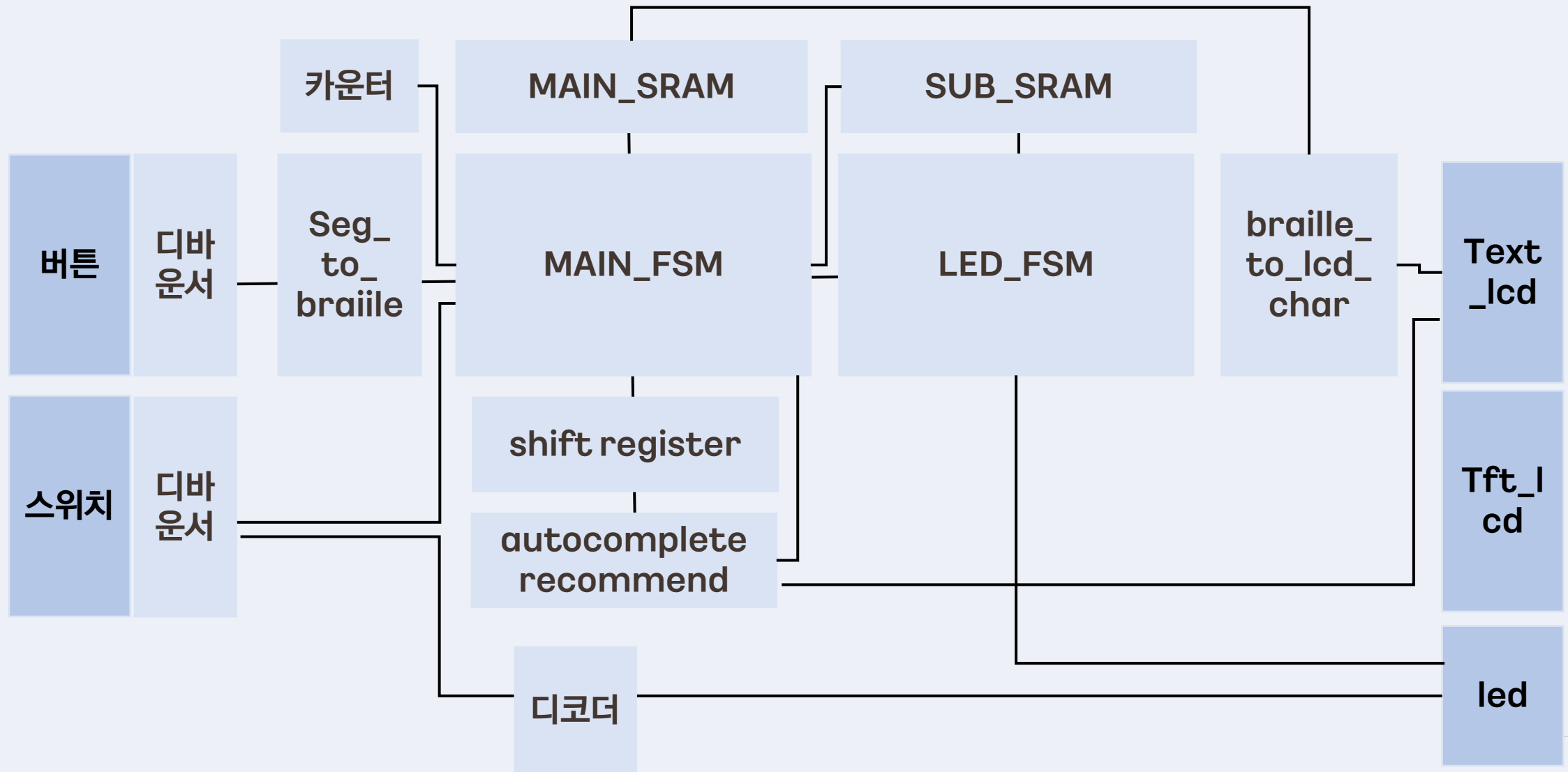
FSM

모듈 간 상호작용 제어

저장장치

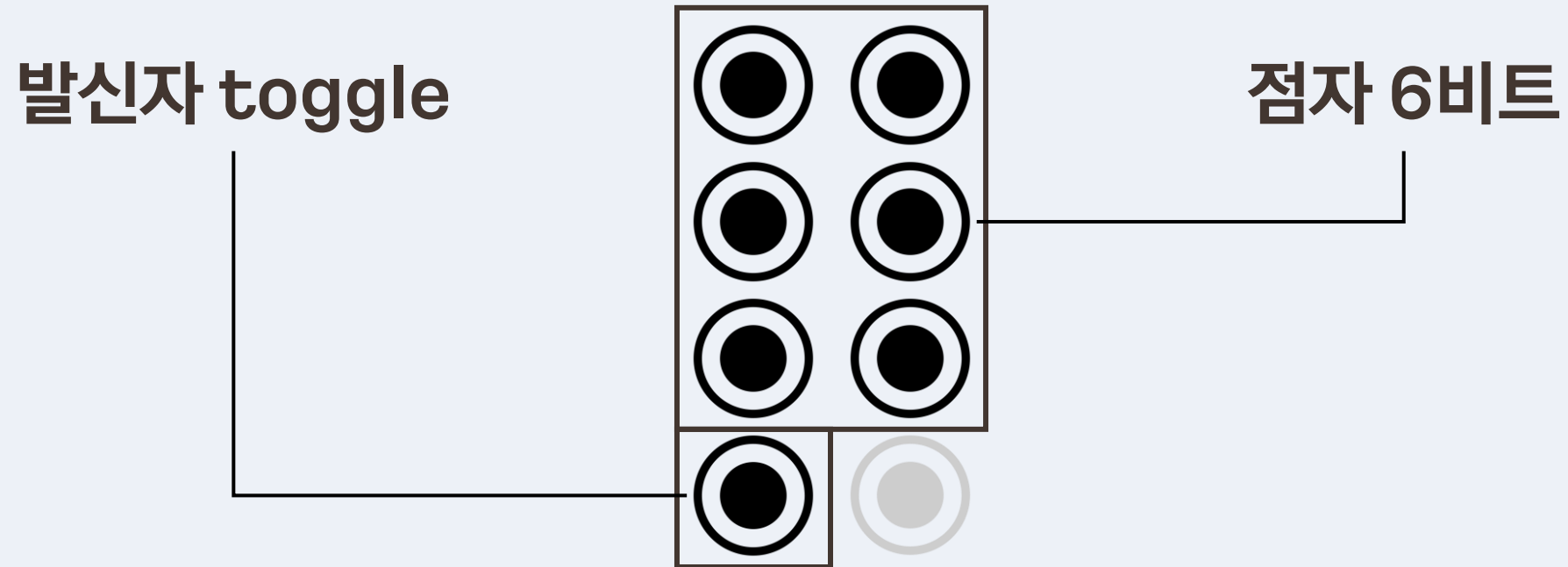
입력된 점자 data 저장

2. 코드 구조 – TOP MODULE



2. 코드 구조 - Button

Button controller



2. 코드 구조 - Button

Debouncer

FPGA 입력 신호의 바운싱 현상

일정 시간동안 동일한 신호가 입력되는지 검사

빠른 주기로 신호가 변하는 바운싱 무시

2. 코드 구조 – Button

Seg_to_Braille

각 버튼 입력 값을 6비트 점자 코드로 변환 -> 점자 데이터 생성

Decoder

입력 값을 one-hot 형식으로 변환

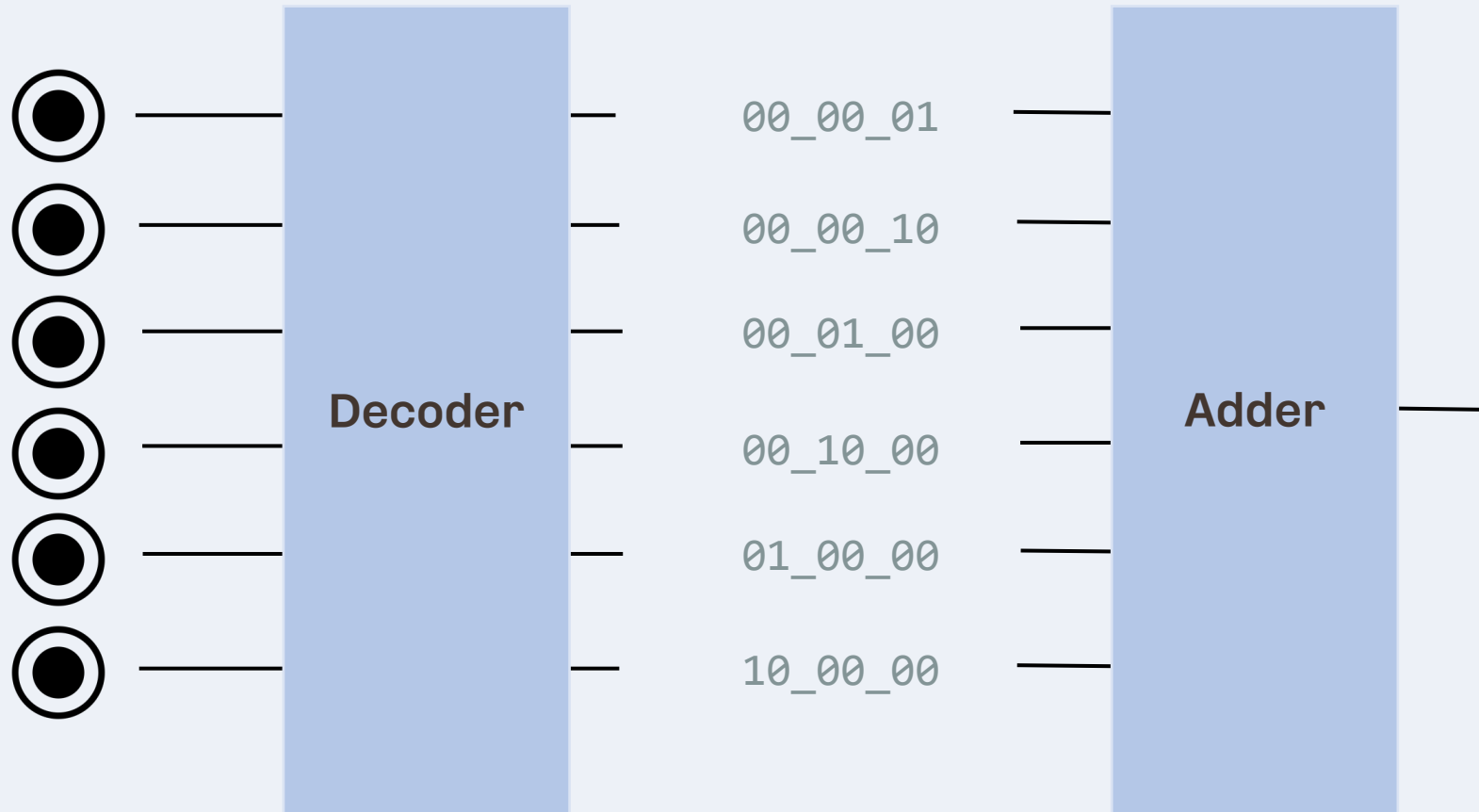
Adder

one-hot 형태의 입력신호 종합

2. 코드 구조 – Button

Seg_to_Braille

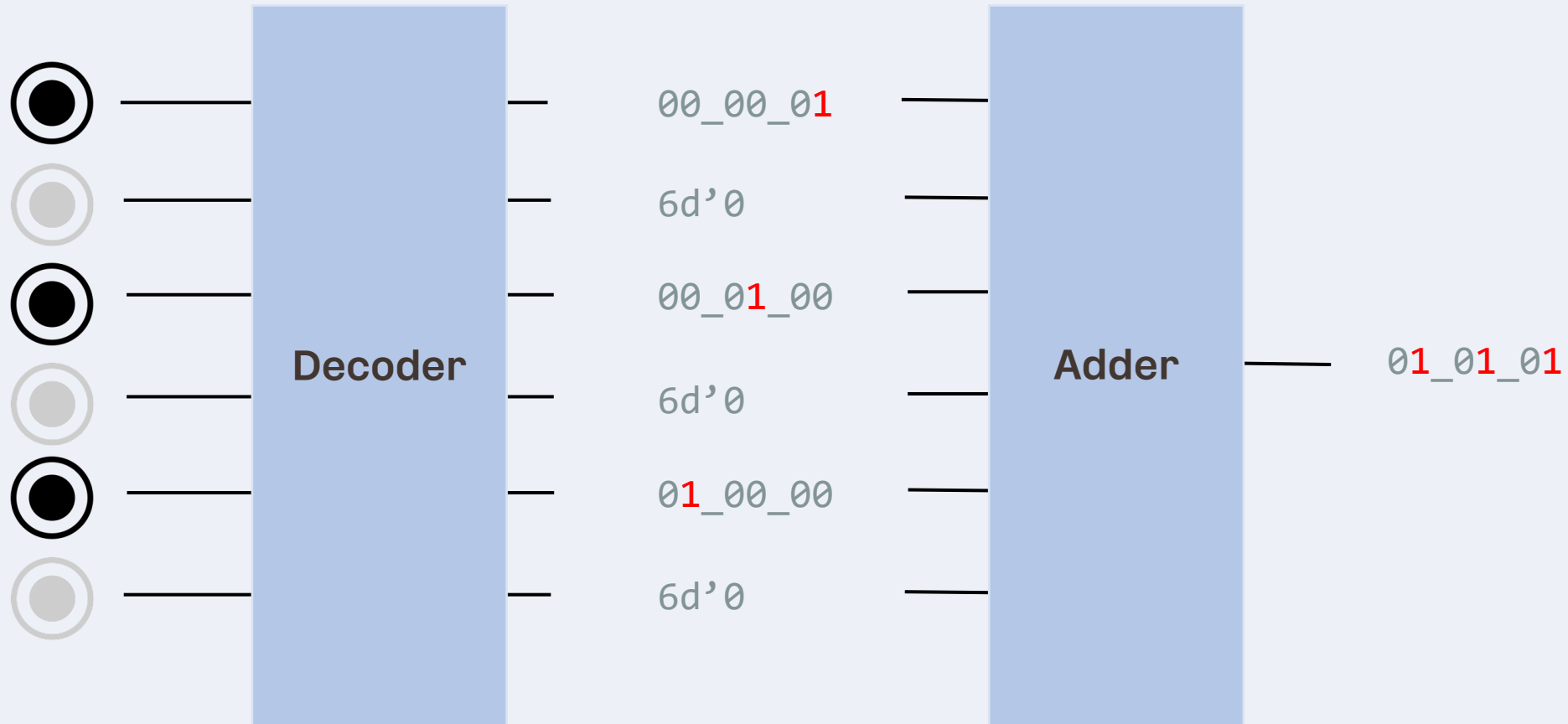
각 버튼 입력 값을 6비트 점자 코드로 변환 -> 점자 데이터 생성



2. 코드 구조 – Button

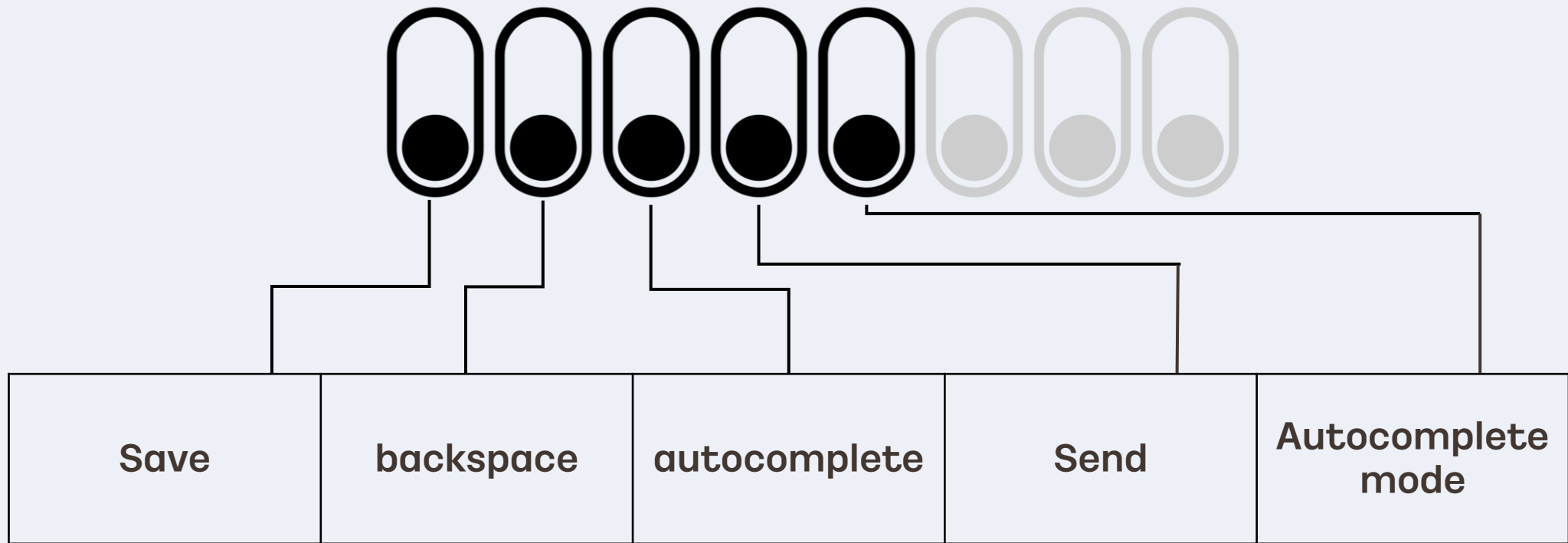
Seg_to_Braille

각 버튼 입력 값을 6비트 점자 코드로 변환 -> 점자 데이터 생성



2. 코드 구조 - Switch

Switch controller



2. 코드 구조 - Text_Lcd

점자 입력 결과를 문자 형태로 표시하는 출력 장치

Braille_
to_lcd_
char

MAIN_SRAM으로부터 6비트 data를 받아, 8비트 문자 코드로 변환

Text_lcd

braille_to_lcd_char로부터 받은 문자 코드를 지정된 위치에 출력

2. 코드 구조 - Text_Lcd

MAIN_SRAM

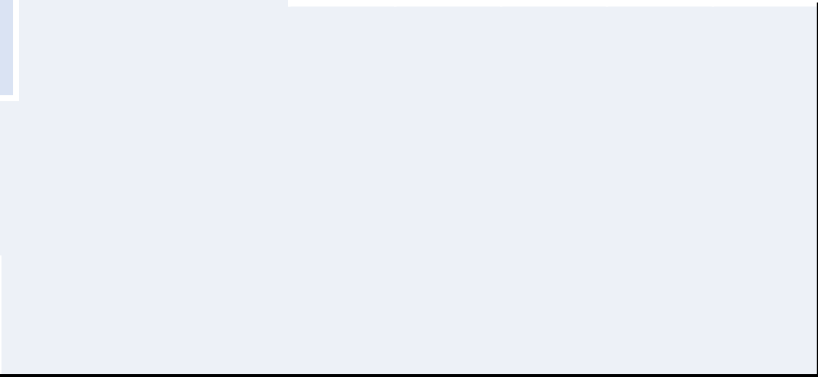
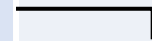
a	:	h	i	
b	:	b	y	e

(실제 형식은 6비트 점자 data)

Text_Lcd

a	:	h	i						
b	:	b	y	e					

자동완성 모듈



2. 코드 구조 - Tft_lcd



TFT LCD
Top

UI 요소

TFT LCD
Controller

TFT LCD 제어

Kakao
Bubble
Pattern

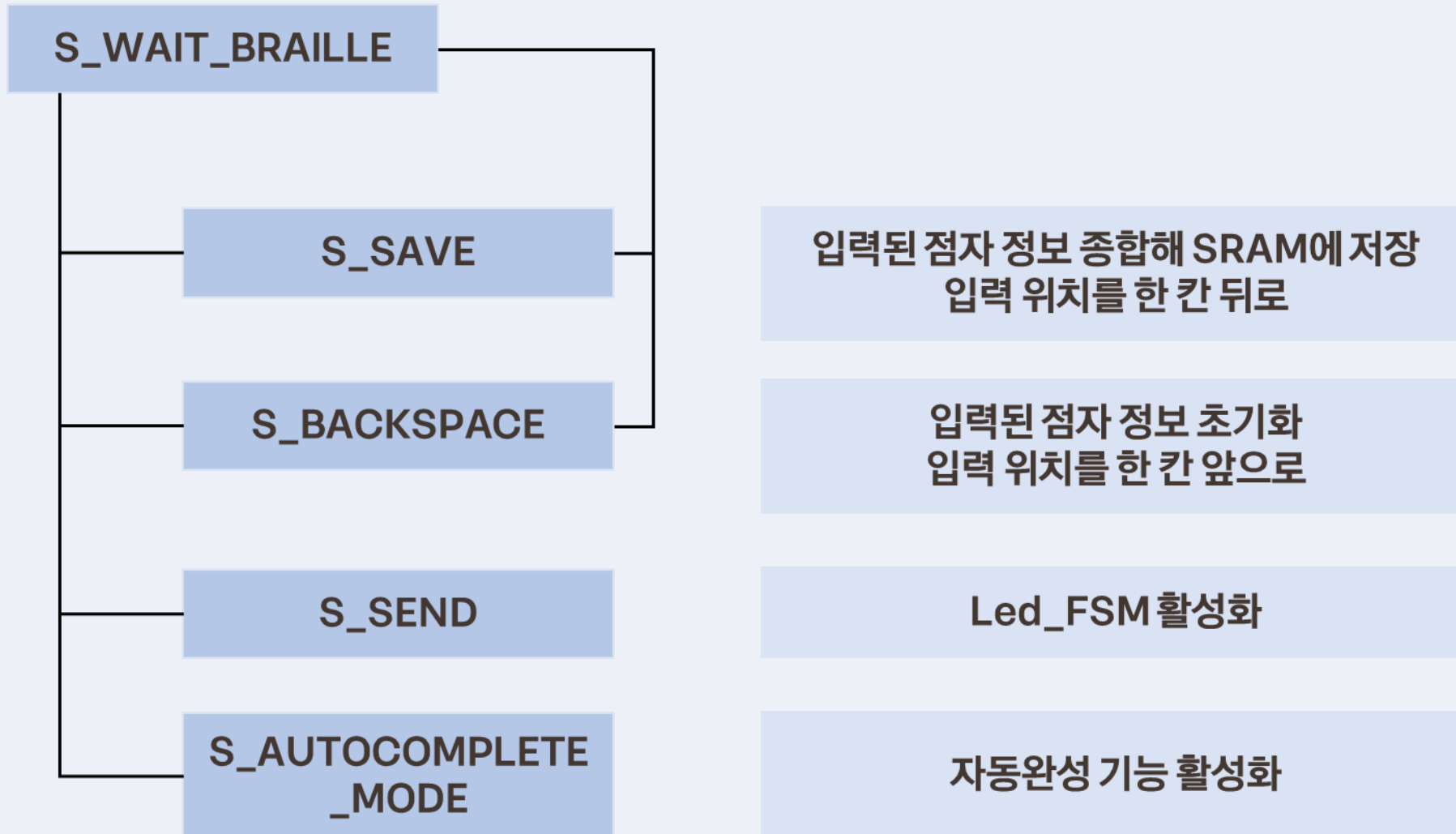
카카오톡 메인 화면에 대한 데이터 저장

2. 코드 구조 – MAIN FSM

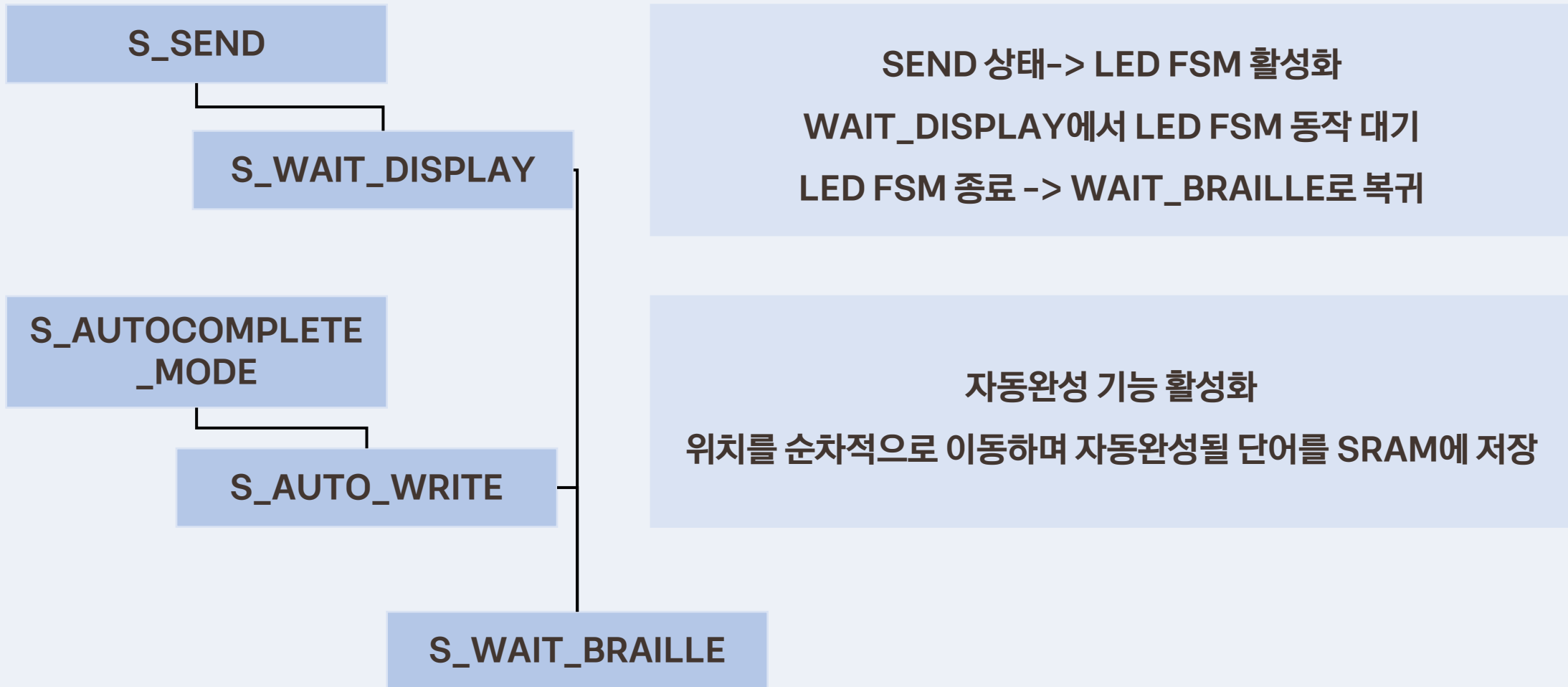
각 입력 장치와 SRAM, LED FSM와 상호작용

S_WAIT_USER	발신자 정보 입력 대기
S_SAVE0	SRAM에 발신자 정보 저장
S_SAVE1	SRAM에 ":" 저장
S_WAIT_BRAILLE	버튼을 통하여 점자 정보를 입력 or 스위치를 통하여 추가 동작에 대한 신호 입력

2. 코드 구조 - MAIN FSM



2. 코드 구조 - MAIN FSM

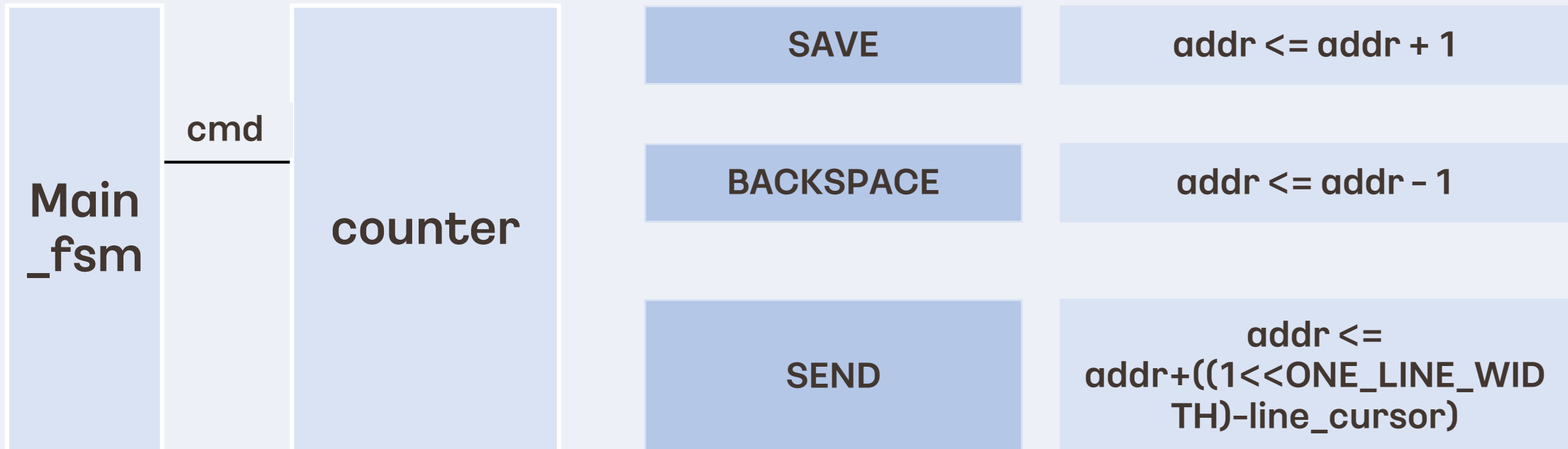


2. 코드 구조 - MAIN FSM

counter

MAIN FSM에서 현재 동작에 해당하는 cmd 신호 counter에 전달
-> counter에서 현재 동작에 맞게 addr을 갱신

```
CMD_IDLE      = 3'b000; //idle  
CMD_SPACE     = 3'b001; //space,  
CMD_BACKSPACE = 3'b010; //backspace  
CMD_SEND      = 3'b011; //send
```

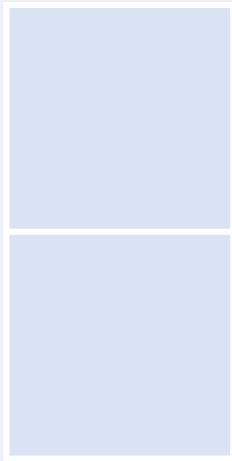


2. 코드 구조 - Autocomplete

Shift register

최근 입력된 2개의 점자에 대한 정보를 기억

Data_in[5:0]



Last_input[11:6]

Last_input[5:0]

Autocomplete_recommend

내장된 단어 중 자동완성이 가능한 단어가 있는지 검사

→ Shift register의 출력값과 앞의 두 자리가
일치하는 단어가 있다면 자동완성 가능

Autocomplete
recommend

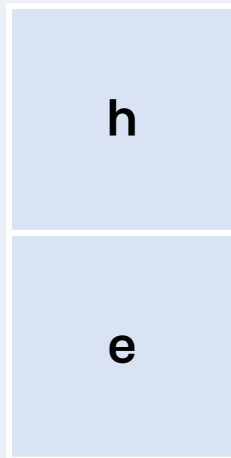
Autocomplete_able

Left_chars[17:0]

2. 코드 구조 - Autocomplete

Shift register

Data_in[5:0]



Last_input[11:6]

Last_input[5:0]

Autocomplete
_recommend



Autocomplete_able

Left_chars[17:0]



2. 코드 구조 - Autocomplete

Autocomplete
_recommend

h	e	l	l	o
l	o	v	e	
y	o	u		
⋮				

Text_lcd

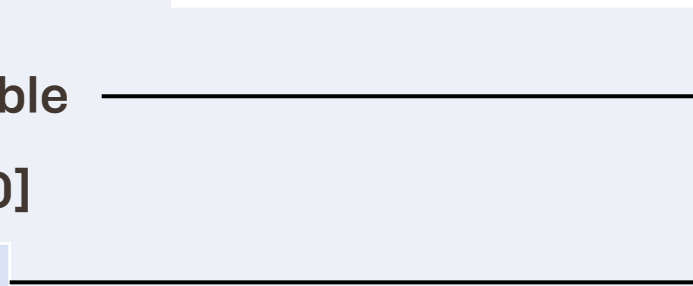
a	:	h	i						
b	:	h	e	l	l	o			

Autocomplete_able

Left_chars[17:0]

l	l	o
---	---	---

Blink_on



2. 코드 구조 – LED FSM

현재 line에 입력된 모든 글자에 대한 점자 정보 순차 출력

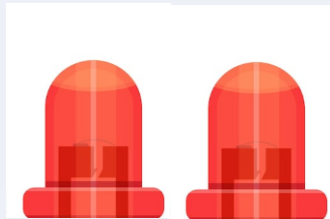
초기 상태 (S_IDLE)	출력 시작 신호 대기, LED OFF 유지
S_WAIT	MAIN FSM에서 신호를 받아 LED FSM 활성화
S_OUTPUT	SRAM에 저장된 현재 인덱스의 6비트 점자 패턴을 LED ON
S_BLINK	LED OFF -> 점멸 효과
S_NEXT	다음 인덱스로 이동, 표시할 점자가 남았는지 확인
S_DONE_HOLD	완료 신호 MAIN FSM에 전달한 후 종료

2. 코드 구조 – LED FSM

LED

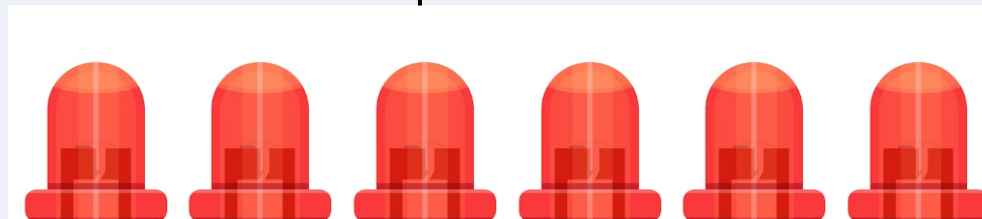
button

발신자 정보(decoder)
a -> 2'b10
b -> 2'b01



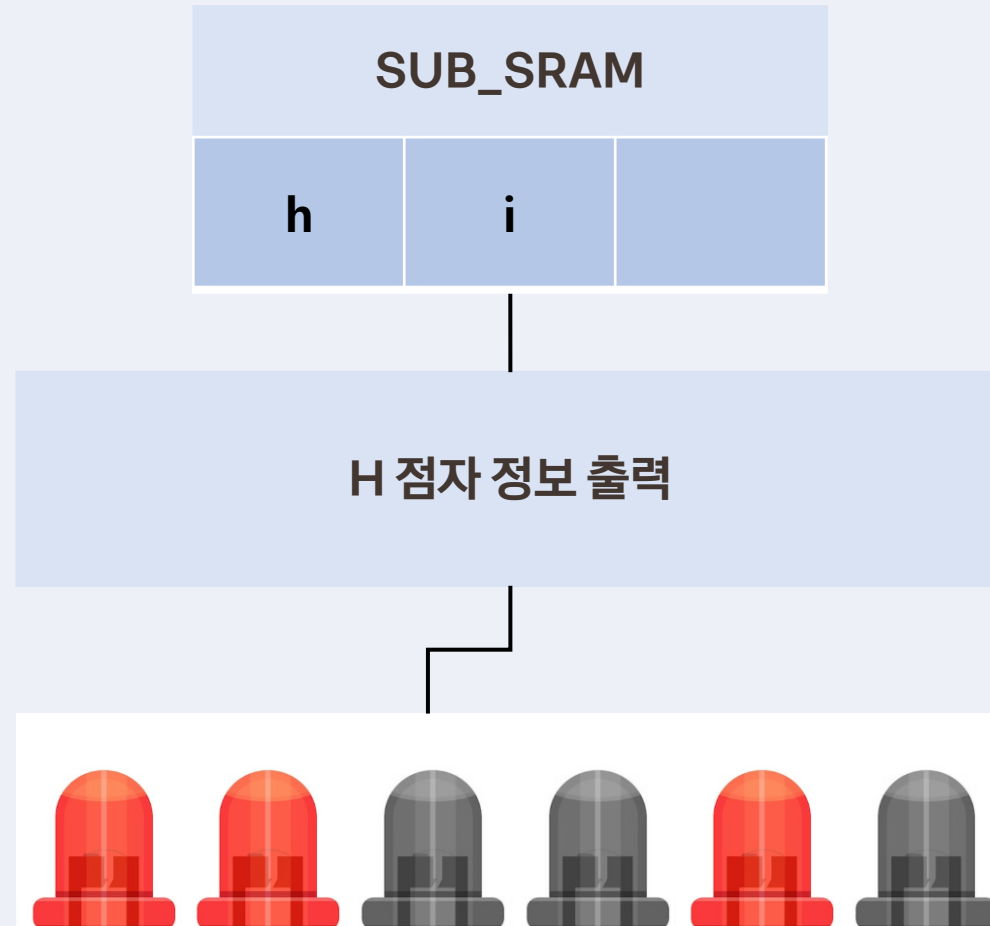
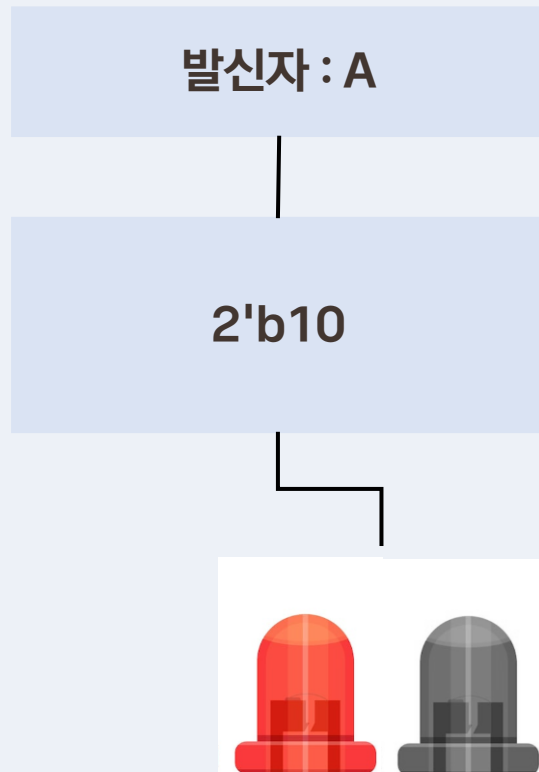
SUB_SRAM

해당 line에 저장된 6비트 점자 정보 순차 출력



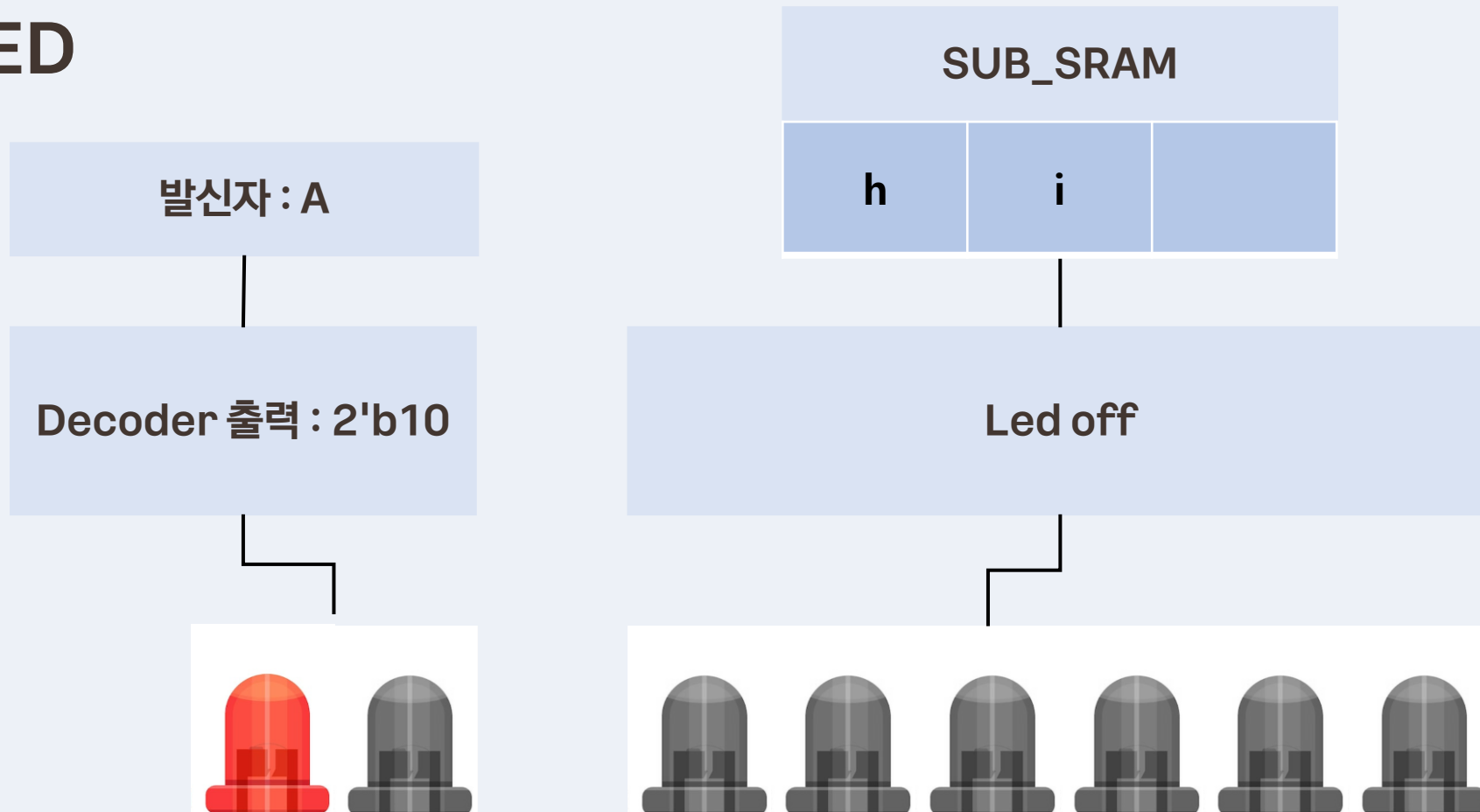
2. 코드 구조 - led fsm

LED



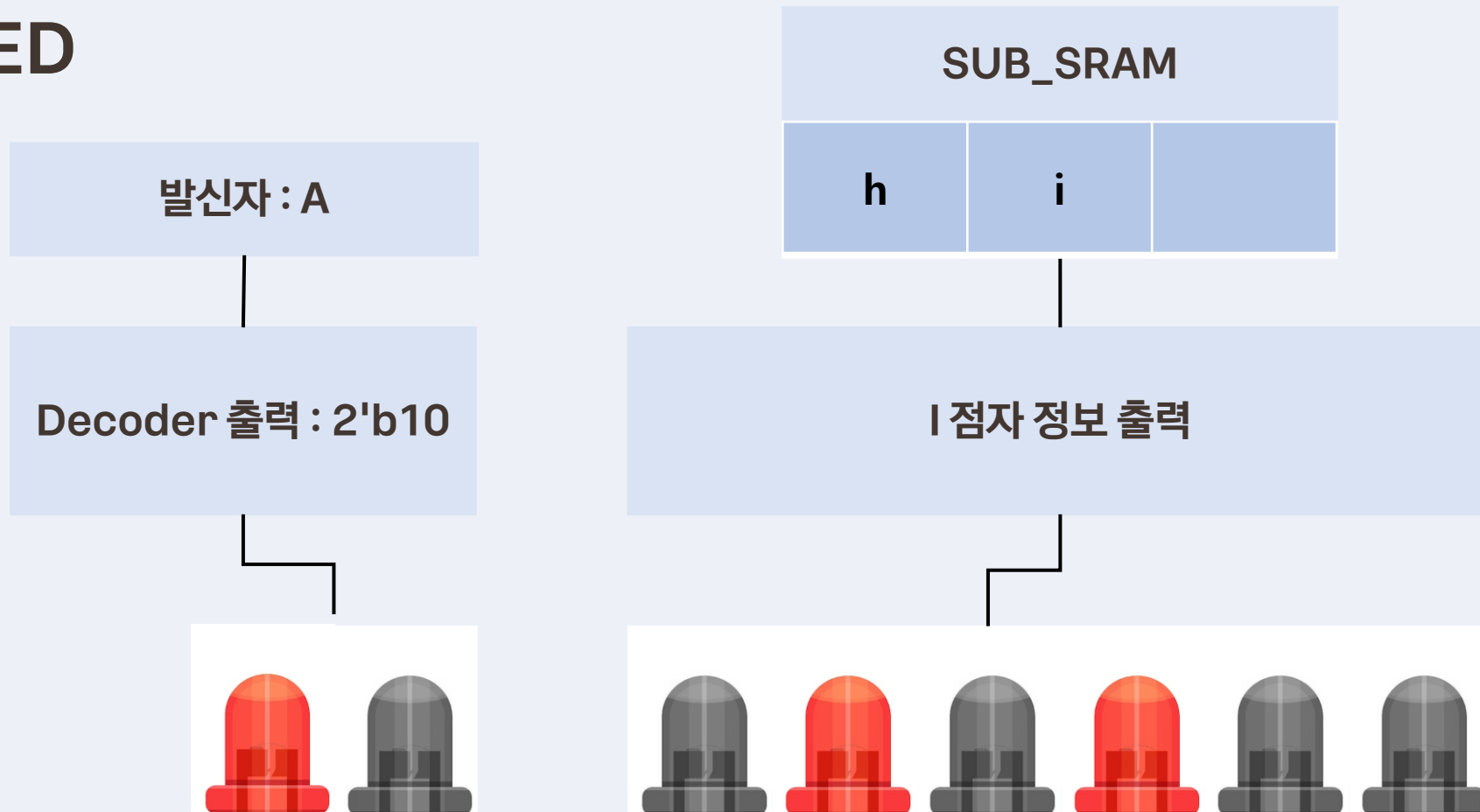
2. 코드 구조 - led fsm

LED



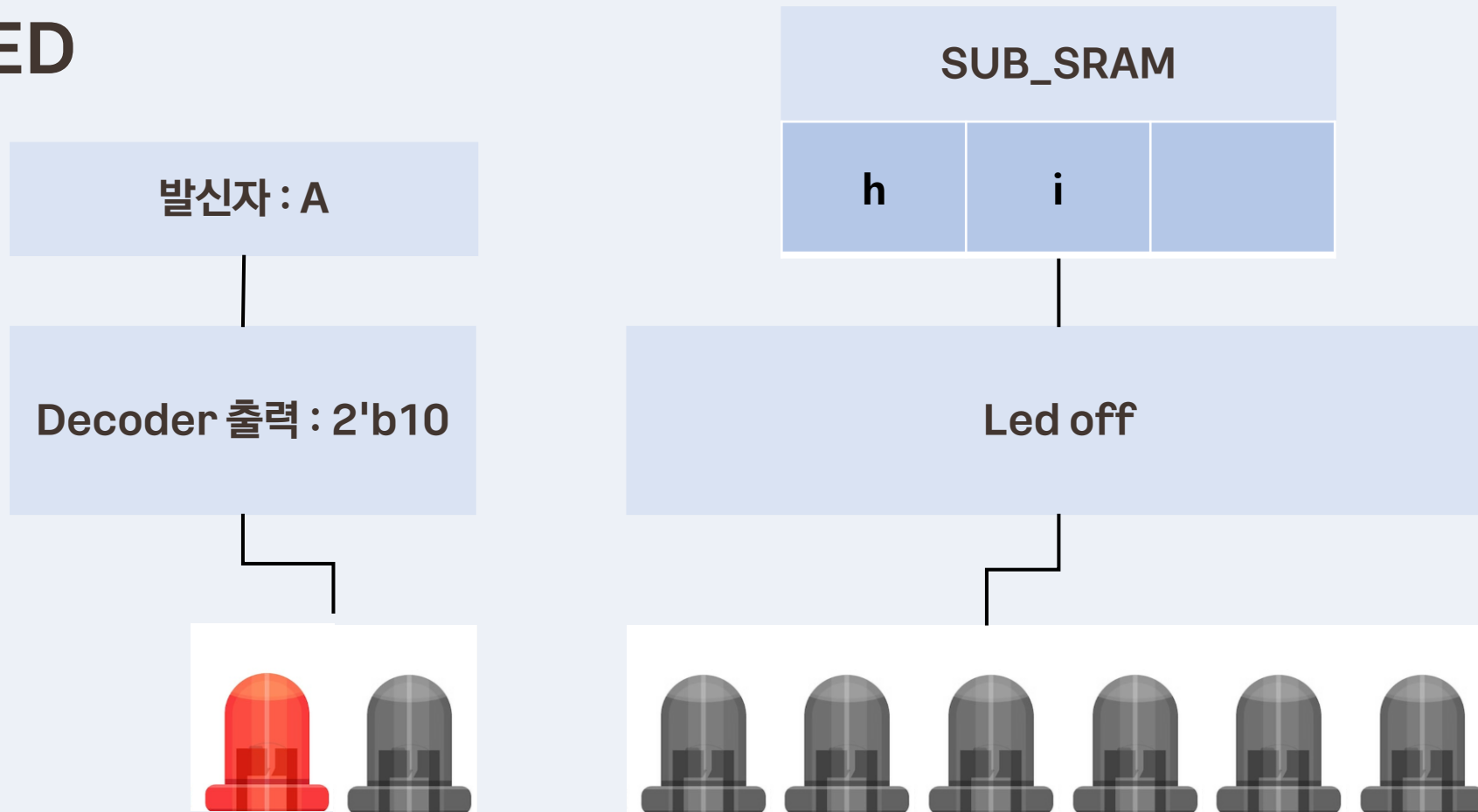
2. 코드 구조 - led fsm

LED



2. 코드 구조 - led fsm

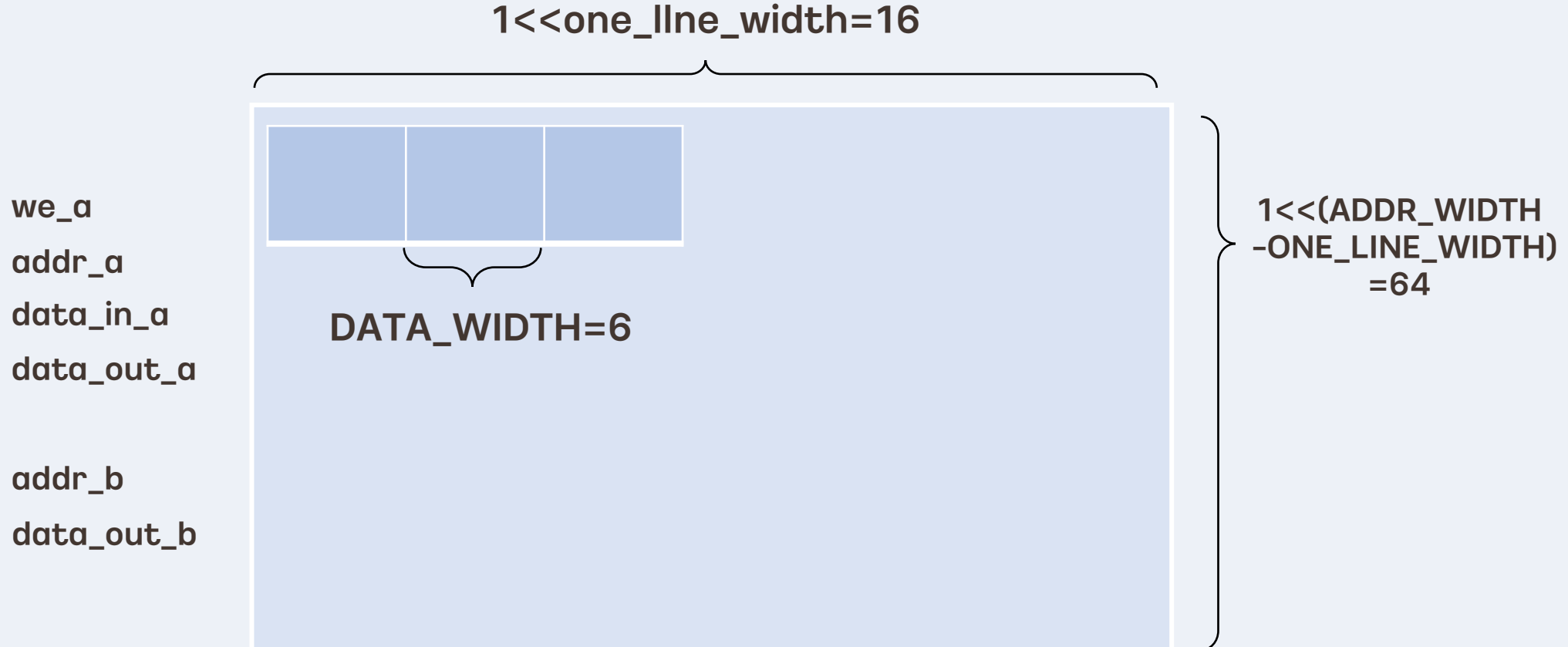
LED



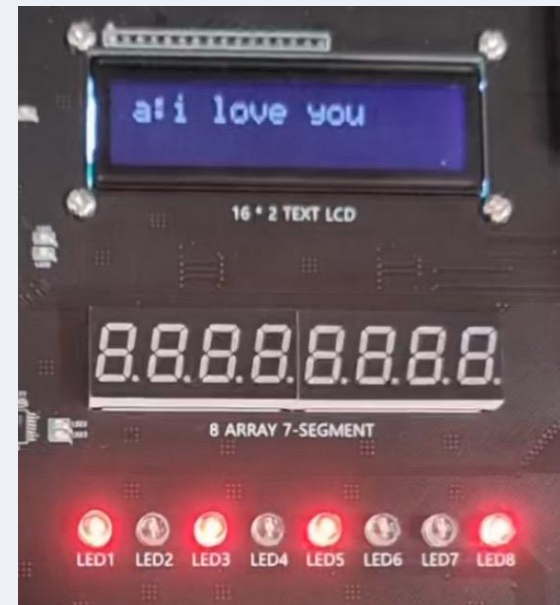
2. 코드 구조 - 저장 장치

SRAM

```
parameter DATA_WIDTH = 6,  
parameter ONE_LINE_WIDTH = 4,  
parameter ADDR_WIDTH = 10
```



3. 출력 및 결과



4. 결론

점자 채팅은 단순 텍스트 입력이 아니라 점자를 기반으로 한 **양방향 소통 구조**

Text lcd만 사용할 경우 출력이 시각 정보에만 의존하게 되어 시각장애인 수신 불가

→ "채팅"이라는 개념이 제한

LED 점자 출력은 입력된 문장을 다시 점자 형태로 반환

→ LED 대신 촉각·청각 등의 감각을 전달하는 장치 연결

→ 시각장애인을 위한 **양방향 소통 시스템**



Thank you!